

Développeur en Intelligence Artificielle

Réaliser la collecte, le stockage et la mise à disposition des données d'un projet en intelligence artificielle

Projet : Mix-Energie, Pipeline de production électrique & empreinte carbone

Candidat	Thibault Merville
Repo du projet	https://github.com/ThibaultMer/mix_energy
Certification	Développeur en Intelligence Artificielle — Niveau 6
Bloc évalué	BC01 — Collecte, stockage et mise à disposition des données
Certificateur	Simplon.co — RNCP37827
Date	20 avril 2026

Repo du projet



Table des matières

1.	Résumé du projet et architecture du pipeline	3
2.	Collecter les données de manière automatisée	5
1)	Contexte et sources de données	5
2)	Script d'extraction	5
3)	Connexion et upload vers Google Cloud Storage	6
4)	Orchestration Airflow	7
3.	Préparation : Nettoyage et Agrégation	10
1)	Inférence dynamique des schémas	10
2)	Règles de typage et nettoyage des données	10
3)	Couche silver dbt.....	11
4)	Couche gold	12
5)	Modèle de préparation pour la prédiction.....	13
6)	Module de prédiction & versioning MLflow.....	13
7)	Récapitulatif des couches dbt et dbt lineage graph.....	15
4.	Base de données	16
1)	Choix technologique : BigQuery (GCP)	16
2)	Schéma physique, chargement CSV vers BigQuery.....	16
3)	Traçabilité des chargements.....	17
4)	Configuration dbt	18
5)	Schémas des tables	19
6)	Conformité RGPD	20
5.	Mettre à disposition les données via une API	21
1)	Architecture REST : FastAPI	21
2)	Schémas Pydantic.....	22
3)	Endpoint de requête générique	23
4)	Sécurisation et CORS	23
5)	Dashboard Streamlit.....	24
6.	Déploiements	25
7.	Infrastructure as Code avec Terraform	26
8.	Conclusions	27
9.	Axes d'améliorations	28

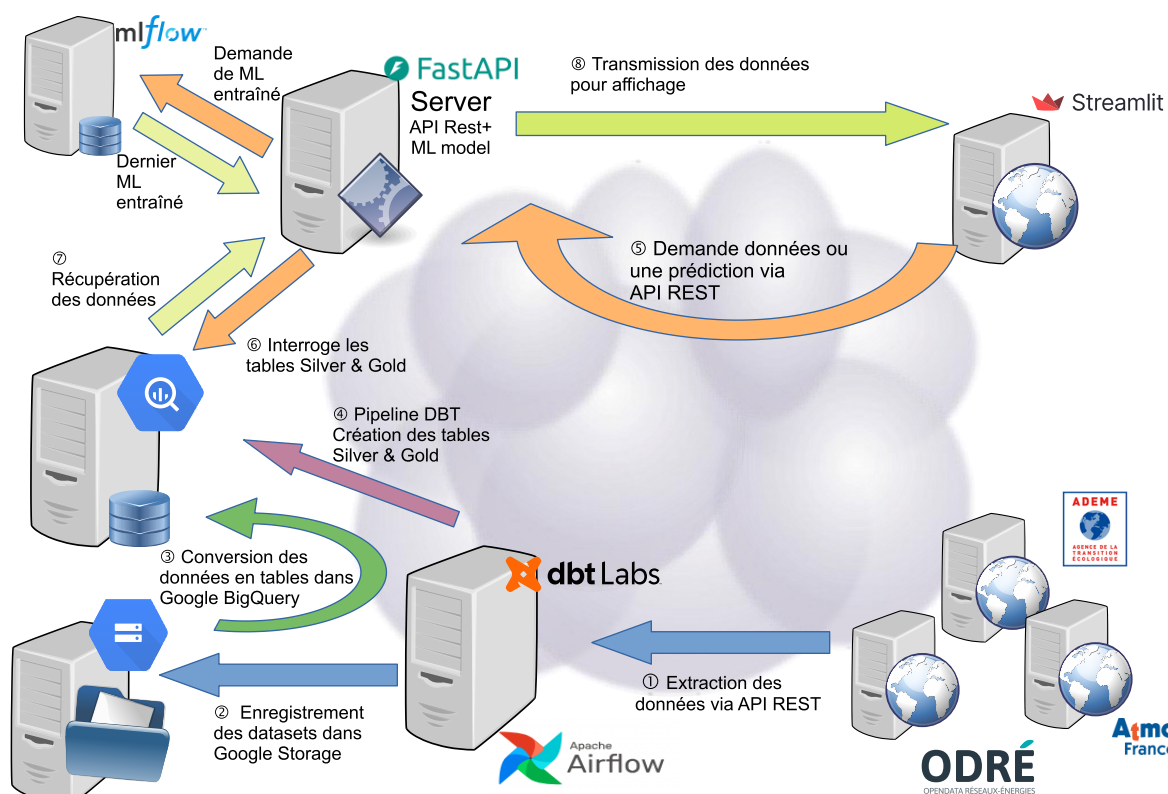
1. Résumé du projet et architecture du pipeline

Le projet Mix-Energie est un pipeline de données qui collecte en quasi-temps-réel les données de production électrique française par source d'énergie (nucléaire, solaire, éolien, gaz, hydraulique...), permet de prédire la production future et expose les données via une API et un front Streamlit.

Pour ce projet, nous avons décidé d'utiliser les différents produits proposés par Google Cloud Platform (GCP), cela permet d'avoir tout intégré au même endroit. Mais il aurait été possible d'utiliser un autre data lake pour l'ingestion des sources et d'une base de données postgreSQL pour les tables.

L'ensemble du projet se trouve sur une VM du produit Compute Engine. Les dockers sont montés et tournent sur la VM.

Voici un schéma représentatif des différents éléments composant le pipeline et leurs interactions :



Le pipeline repose sur cinq couches fonctionnelles enchaînées :

- ▶ Collecte automatisée (Airflow) : DAGs Python orchestrant les appels à l'API ODRE/éCO2mix et le dépôt des CSV dans un bucket de Google Cloud Storage.
- ▶ Ingestion & nettoyage (ingest_dbt) : scripts Python détectant les séparateurs, corrigeant les valeurs nulles et inférant dynamiquement les schémas BigQuery.
- ▶ Transformation & agrégation (dbt) : modèles SQL en couches silver (nettoyage) et gold (KPIs journaliers, taux CO₂ pondéré, table permettant l'apprentissage du modèle de prédiction).
- ▶ Stockage (BigQuery) : entrepôt de données GCP hébergeant les tables brutes, intermédiaires et exposées.
- ▶ Exposition (FastAPI) : API REST sécurisée avec documentation OpenAPI intégrée, exposant les données et les prédictions de consommation.

Trois éléments supplémentaires complètent ce pipeline :

- ▶ L'affichage des données gold sur un front grâce à l'outil streamlit
- ▶ L'apprentissage d'un modèle stocker sur mlflow permettant la prédiction
- ▶ La mise en place d'un Terraform pour

2. Collecter les données de manière automatisée

1) Contexte et sources de données

Le projet ingère quatre jeux de données distincts depuis l'API open data d'éCO2mix (RTE/ODRE), accessibles depuis l'URL de base :

```
https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets/
```

Dataset ID	Contenu	Fréquence de mise à jour
eco2mix-national-tr	Production/conso nationale temps réel	Toutes les 15 min
eco2mix-national-cons-def	Données nationales consolidées	Quotidienne
eco2mix-regional-tr	Production régionale temps réel	Toutes les 15 min
eco2mix-regional-cons-def	Données régionales consolidées	Quotidienne
Base Carbone ADEME	Facteurs d'émission CO2 par filière de production (gCO2e/kWh)	Annuelle

Deux autres sources de données sont également récupérées mais pas utilisées à ce stade :

Dataset ID	Contenu	Fréquence de mise à jour
Open-Meteo	Données météo historique et prédiction jusqu'à j+16	Horaire
Atmo Data	Indice de la qualité de l'air par commune	Quotidienne

2) Script d'extraction

La fonction `retrieve_csv()` effectue un appel HTTP GET vers l'endpoint d'export CSV de l'API ODRE, avec gestion exhaustive des codes d'erreur HTTP.

Voici la fonction présente dans le fichier `eco2mix_ingest.py` :

```
def retrieve_csv(
    dataset_id: str,
    delimiter: str = ";",
    list_sep: str = ",",
    quote_all: bool = False,
    with_bom: bool = True,
):
    req_url = base_url + dataset_id + "/exports/csv"

    params = {
```

```

        "delimiter": delimiter,
        "list_separator": list_sep,
        "quote_all": quote_all,
        "with_bom": with_bom,
    }

    result = __perform_request(req_url=req_url, params=params)

    if result.status_code == 200:
        if (
            "content-type" in result.headers
            and result.headers["content-type"].split(";")[0] == "text/csv"
        ):
            logger.info("CSV file of the dataset {} retrieved", dataset_id)
            return result.content
        else:
            logger.error(
                "Unexpected data received : {}".format(result.headers["content-
type"])
            )
    else:
        return None

```

La fonction `retrieve_csv()` renvoie directement les bytes du CSV sans passer par le disque local. La vérification du Content-Type garantit qu'on ne stocke pas une réponse d'erreur JSON à la place du fichier attendu.

La fonction `__perform_request()`, permet de réaliser la requête avec l'url de la source et de vérifier les codes réponses, elle permet d'avoir des logs sur la connexion en question pour tracer d'éventuels problèmes.

3) Connexion et upload vers Google Cloud Storage

Le module `gcp_utils.py` gère l'authentification GCP via un compte de service et l'upload du CSV dans le bucket :

```

def connect_to_bucket() -> storage.Bucket:

    # Create the credential used to authenticate
    json_credential_file = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")
    project_id = os.getenv("PROJECT_ID")

    # Si la variable d'environnement n'est pas définie, utiliser le chemin par défaut fourni
    if not json_credential_file:
        json_credential_file = os.path.join(
            os.path.dirname(__file__), "data/gcp/mix-energie-gcp-8cd25d0352aa.json"
        )
        get_logger().warning(
            f"GOOGLE_APPLICATION_CREDENTIALS non défini, utilisation du chemin local : {json_credential_file}"
        )

```

```

    )
    if not os.path.exists(json_credential_file):
        get_logger().error(
            f"Fichier de credentials introuvable : {json_credential_file}"
        )
    return None

    credentials = service_account.Credentials.from_service_account_file(
        json_credential_file
    )

    get_logger().info("Connection to the project ")

    try:
        client = storage.Client(project=project_id, credentials=credentials)
    except Exception as e:
        get_logger().error("Fail to connect to project {} : {}".format(project_id, e))
        return None

    try:
        bucket = client.get_bucket("mix-energie-bucket")
    except Exception as e:
        get_logger().error(
            "Fail to retrieve bucket 'mix-energie-bucket' in project {} : {}".format(
                project_id, e
            )
        )
    return None

    return bucket

```

```

def upload_data_in_bucket(bucket, data, dataset):
    get_logger().info("Load data on the bucket : {}".format(dataset))
    blob = bucket.blob(dataset + ".csv")
    blob.upload_from_string(data)

```

La fonction `connect_to_bucket()` récupère la clef JSON local qui a été générée depuis un service account de GCP. Elle effectue ensuite la connexion au bucket et renvoie celui-ci afin qu'il puisse être utilisé par la suite.

4) Orchestration Airflow

Prenons en exemple le dag « `dag_eco2mix_national_tr.py` ». Ce dag récupère les données nationales de production en temps réel. Les données sont actualisées toutes les 15 minutes.

Le DAG orchestre le pipeline en 5 tâches, trois qui se succèdent et 2 en parallèle après les trois premières, planifiées via un cron multi-plages :

```

@dag(
    dag_id="dag_eco2mix_national_tr",
    description="Ingestion eco2mix national tr vers GCS.",
    start_date=datetime(2026, 1, 1),

```

```

schedule=MultipleCronTriggerTimetable(
    "30,45 9 * * 1-5",
    "0,15,30,45 10-17 * * 1-5",
    "0,15,30 18 * * 1-5",
    timezone="Europe/Paris",
),
catchup=False,
tags=["eco2mix", "ingestion"],
)

```

J'ai utilisé une cron multi-plage pour une raison particulière. Afin de réduire les coûts de la VM, celle-ci n'est allumée qu'entre 9h et 19h. Pour éviter que les DAGs ne remontent en erreurs, je limite l'exécution des DAGs entre 9h30 et 18h30. Je commence à 9h30 afin d'éviter de lancer un dag avant que la VM ne soit entièrement lancée, et j'arrête à 18h30 pour que les DAGs puissent s'exécuter complètement avant l'arrêt de la machine.

Dans ce DAGs, les tâches sont dépendantes des autres pour éviter d'essayer de charger des données alors qu'on ne récupère pas le bucket. Les deux dernières, concernant la création de tables avec dbt, elles dépendent de l'ingestion des données. Ces créations de tables étant indépendante, il est possible de réaliser ces tâches en parallèle.

```

def dag_eco2mix_national_tr():
    @task(task_id="check_bucket_connection")
    def check_bucket_connection() -> str:

    @task(task_id="ingest_csv_to_bucket")
    def ingest_csv_to_bucket(bucket_name: str, dataset_id: str) -> None:

    @task(task_id="transfer_csv_to_bigquery")
    def transfer_csv_from_bucket_to_bigquery(file_prefix: str) -> None:

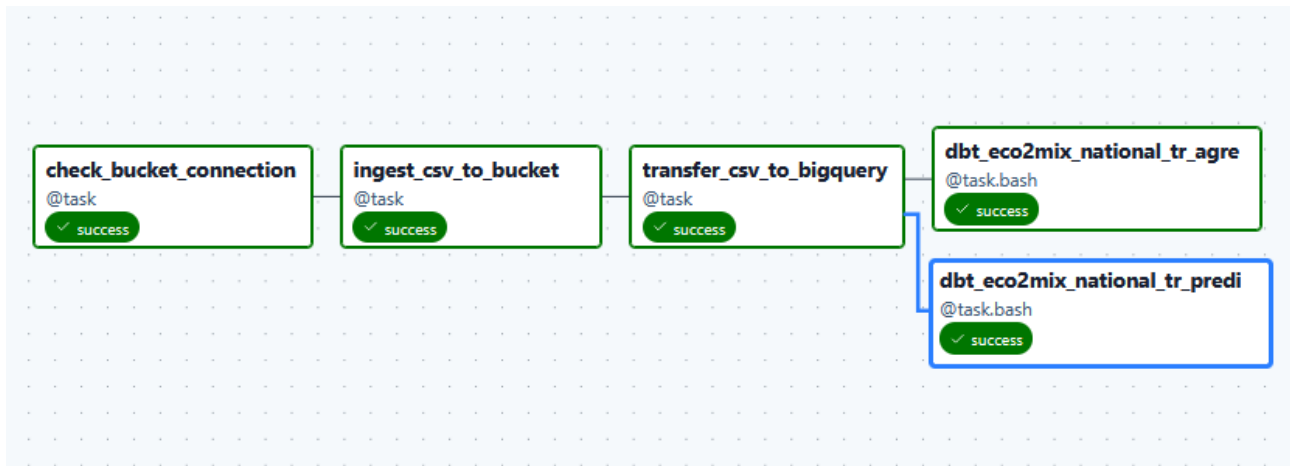
    @task.bash(task_id="dbt_eco2mix_national_tr_agre")
    def dbt_nat_tr_agre():

    @task.bash(task_id="dbt_eco2mix_national_tr_predi")
    def dbt_nat_tr_predi():

    (
        check_bucket_connection_task
        >> ingest_csv_to_bucket_task
        >> transfer_csv_from_bucket_to_bigquery_task
        >> [dbt_eco2mix_national_tr_agre_task, dbt_eco2mix_national_tr_predi_task]
    )

```


Voici une vue du dag dans Airflow :



check_bucket_connection permet de vérifier qu'on accède bien au bucket.

ingest_csv_to_bucket permet de récupérer le csv et de l'enregistrer dans le bucket.

transfer_csv_to_bigquery permet de récupérer le csv dans le bucket et d'enregistrer les données dans une table bigquery.

Les deux tâches dbt permettent de manipuler la table créée précédemment pour obtenir des tables exploitables pour les Data Analyst ou les Data Scientist.

3. Préparation : Nettoyage et Agrégation

Lors de la définition du projet, nous voulions mettre en place un pipeline ELT avec l'ensemble des transformations gérées par DBT.

Cependant, nous avons rencontré des problèmes pour transférer les fichiers du bucket vers les tables BigQuery, nous avons dû pré-nettoyer certains fichiers avant de les intégrer dans les tables. Notre pipeline est plutôt de type ETLT, nous faisons une extraction, nous réalisons des transformations pour ensuite les charger dans une base de données puis nous transformons de nouveau les données pour avoir des tables finales exploitables.

1) Inférence dynamique des schémas

Plutôt que de définir statiquement un schéma complet pour chaque source, le projet infère les types BigQuery à partir d'un échantillon du CSV. Chaque type de fichier dispose d'une fonction de résolution dédiée. Voici un détail du code provenant du fichier `bigquery_schema_generator.py` :

```
def create_bigquery_schema_for_file(
    filename: str, df: pd.DataFrame
) -> list[bigquery.SchemaField]:
    """
    Sélectionne la boucle de schema selon le prefixe du nom de fichier CSV.
    """
    filename_lower = filename.lower()

    if filename_lower.startswith("eco2mix"):
        return create_eco2mix_schema(df)
    if filename_lower.startswith("meteo"):
        return create_meteo_schema(df)
    if filename_lower.startswith("air_quality"):
        return create_air_quality_schema(df)
    return create_generic_schema(df)
```

Comme indiqué plus haut, les datasets sur la météo et la qualité de l'air ne sont pas encore exploités, mais le code pour les intégrer est déjà existant.

La détection par préfixe de nom de fichier (`eco2mix*`, `meteo*`, `air_quality*`) découple la logique métier de l'orchestration. En cas d'échec d'inférence, un schéma de secours tout STRING est appliqué pour ne jamais bloquer le pipeline.

2) Règles de typage et nettoyage des données

La résolution de type pour les fichiers `éCO2mix` illustre les règles de nettoyage appliquées. :

```
def _resolve_eco2mix_type(column_name: Any, dtype: Any) -> str:
    col_str = str(column_name).lower()

    if "time" in col_str or "timestamp" in col_str or "date_heure" in col_str:
        return "TIMESTAMP"
    if "date" in col_str and "heure" not in col_str:
        return "DATE"
    if "heure" in col_str:
        return "STRING"
```

```

    if "libelle_region" in col_str or "nature" in col_str or "perimetre" in
col_str:
        return "STRING"
    if pd.api.types.is_bool_dtype(dtype):
        return "BOOLEAN"
    if pd.api.types.is_integer_dtype(dtype):
        return "INTEGER"
    if pd.api.types.is_float_dtype(dtype):
        return "FLOAT"
    if pd.api.types.is_datetime64_any_dtype(dtype):
        return "TIMESTAMP"
    if pd.api.types.is_string_dtype(dtype) or dtype == "object":
        return "STRING"

return "STRING"

```

Ci-dessous un tableau récapitulatif de la génération du schéma pour les fichiers eco2mix

Colonne source	Type BigQuery	Règle appliquée
date_heure	TIMESTAMP	Détection par nom contenant 'date_heure'
date	DATE	Colonne 'date' sans 'heure'
heure	STRING	Format HH:MM non parsable en TIME
consommation, gaz, nucéaire...	FLOAT	Valeurs numériques avec décimales
libelle_region	STRING	Règle explicite par nom de colonne
Valeurs vides / NA / ND / -	(NULL)	Listées dans NULL_MARKERS

3) Couche silver dbt

Le modèle dbt silver eco2mix_national_cons_def_histo.sql réalise le premier nettoyage au niveau SQL et permet de créer les tables d'historisation des données.

Sélection explicite des colonnes utiles, renommage normalisé, et chargement incrémental pour ne traiter que les nouvelles dates.

```

{{ config(
    alias='eco2mix_national_cons_def_histo',
    materialized='incremental',
    unique_key='date'
) }}

```

```

SELECT
perimetre      AS perimetre,
nature         AS nature,
date           AS date,
heure          AS heure,
date_heure     AS date_heure,
consommation   AS consommation,

```

```

prevision_j1    AS prevision_j1,
prevision_j     AS prevision_j,
fioul           AS fioul,
charbon        AS charbon,
gaz            AS gaz,
nucleaire      AS nucleaire,
eolien         AS eolien,
solaire        AS solaire,
hydraulique    AS hydraulique,
pompage        AS pompage,
bioenergies    AS bioenergies,
ech_physiques  AS ech_physiques,
taux_co2       AS taux_co2
FROM {{source('nat_source', 'eco2mix_national_cons_def')}}

{% if is_incremental() %}
    WHERE date > (SELECT MAX(date) FROM {{ this }})
{% endif %}

```

La matérialisation incremental avec `unique_key='date'` garantit qu'un rechargement complet de la source ne duplique pas les données. Seules les dates postérieures à la dernière présente dans la table cible sont insérées. Cela limite les coûts de requête BigQuery et les temps d'exécution dbt lors des runs quotidiens.

4) Couche gold

Le modèle dbt gold `nat_cons_agre_j.sql` agrège les données demi-horaires de consommation nationale en valeurs journalières. La conversion d'énergie (MW en demi-heure → MWh/jour) et le calcul du taux de CO₂ pondéré illustrent les règles métier implémentées.

```

{{ config(
    alias='nat_cons_agre_j',
    materialized='incremental',
    unique_key='date') }}

select
perimetre as perimetre,
nature as nature,
DATE(date) as date,
EXTRACT(YEAR FROM date) AS annee,
EXTRACT(MONTH FROM date) AS mois,
EXTRACT(DAY FROM date) AS jour,
sum((consommation - pompage - ech_physiques)) * 0.5 as production,
sum(consommation) * 0.5 as consommation,
sum(prevision_j1) * 0.25 as prevision_j1,
sum(prevision_j) * 0.25 as prevision_j,
sum(fioul) * 0.5 as fioul,
sum(charbon) * 0.5 as charbon,
sum(gaz) * 0.5 as gaz,
sum(nucleaire) * 0.5 as nucleaire,
sum(eolien) * 0.5 as eolien,

```

```

sum(solaire) * 0.5 as solaire,
sum(hydraulique) * 0.5 as hydraulique,
sum(pompage) * 0.5 as pompage,
sum(bioenergies) * 0.5 as bioenergies,
sum(ech_physiques) * 0.5 as ech_physiques,
sum(taux_co2 * (consommation - pompage - ech_physiques)) /
NULLIF(sum((consommation - pompage - ech_physiques)), 0) AS taux_co2
from {{ ref('eco2mix_national_cons_def_histo') }}
{% if is_incremental() %}
    WHERE date > (SELECT MAX(date) FROM {{ this }})
{% endif %}
group by perimetre, nature, date, annee, mois, jour
order by date asc

```

Les données brutes éCO2mix sont publiées toutes les 30 minutes en MW moyens. Pour obtenir des MWh journaliers, on somme les 48 demi-heures puis on multiplie par 0,5 (durée de chaque intervalle en heure). Le taux de CO₂ est une moyenne pondérée par la production nette — on utilise NULLIF(sum(...), 0) pour éviter une division par zéro.

5) Modèle de préparation pour la prédiction

Le modèle dbt gold nat_tr_predi.sql prépare les features temporelles nécessaires au modèle de machine learning en décomposant le timestamp en composantes discrètes :

```

{{ config(
    alias='nat_tr_predi',
    materialized='table'
) }}

SELECT
consommation,
prevision_j1,
prevision_j,
EXTRACT(YEAR FROM date_heure) AS year,
EXTRACT(MONTH FROM date_heure) AS month,
EXTRACT(DAY FROM date_heure) AS day,
EXTRACT(HOUR FROM date_heure) AS hour,
EXTRACT(MINUTE FROM date_heure) AS minute
FROM {{ source('nat_source', 'eco2mix_national_tr') }}
WHERE EXTRACT(YEAR FROM date_heure) = EXTRACT(YEAR FROM CURRENT_DATE())
and consommation is not null
order by date_heure asc

```

La décomposition temporelle (year, month, day, hour, minute) transforme le timestamp en features numériques directement exploitables par un modèle de régression. Le filtre WHERE sur l'année courante limite le volume de données transmises au module de prédiction tout en conservant la saisonnalité. La condition consommation IS NOT NULL élimine les demi-heures non encore publiées par RTE.

6) Module de prédiction & versioning MLflow

Le module de prédiction est structuré pour garantir la traçabilité, la reproductibilité et la gestion des versions des modèles de machine learning. Il s'appuie sur une architecture modulaire : gestion des données, prétraitement, entraînement, et log/chargement des modèles.

Architecture du module

Gestion des données : Le module [data.py](#) centralise la connexion à BigQuery, le chargement et la préparation des jeux de données, avec des classes d'énumération pour distinguer les zones (régionale/nationale).

Prétraitement : Le pipeline de transformation est construit via scikit-learn, combinant imputation et normalisation :

```
def build_pipeline() -> Pipeline:
    num_pipe = Pipeline([
        ("knn_imp", KNNImputer(n_neighbors=5)),
        ("scaler", StandardScaler())
    ])
    preprocessor = ColumnTransformer([
        ("numeric", num_pipe, make_column_selector(dtype_include="number"))
    ]).set_output(transform="pandas")
    return preprocessor
```

Ce pipeline assure que les valeurs manquantes sont imputées et que les variables numériques sont normalisées, ce qui est essentiel pour la robustesse des modèles.

Modélisation : La classe Energypredict dans le fichier models.py encapsule la logique de création, d'entraînement et de gestion du modèle :

```
class Energypredict:
    def __init__(self, mllogger: pmllog.MlLog):
        self.__model = None
        self.__mllogger = mllogger

    def create_model(self) -> None:
        self.__model = LinearRegression()
```

L'injection du logger (local ou MLflow) permet d'abstraire la gestion du versioning et du stockage du modèle.

Entraînement et versioning avec MLflow

L'entraînement est orchestré dans train.py, où l'on choisit dynamiquement le logger :

```
with MLFlowLogger(savename) as mlogger:
    predicteng = Energypredict(mlogger)
    # Chargement des données, entraînement, log des métriques...
```

L'utilisation du context manager garantit que chaque run est bien enregistré dans MLflow, avec début et fin de session.

Le logger MLflow (voir [mllogs.py](#)) gère l'enregistrement, le suivi des hyperparamètres, des métriques et la récupération des modèles :

```
class MLFlowLogger(MLog):
    def save_model(self, model: BaseEstimator, name: str):
        model_name = f"{self.savename}_{name}"
        mlflow.sklearn.log_model(
            sk_model=model, name=model_name, registered_model_name=model_name
        )
    def log_hyperparams(self, params: dict):
        mlflow.log_params(params)
    def log_metrics(self, metrics):
        mlflow.log_metrics(metrics)
```

Chaque modèle est versionné et enregistré sur le serveur MLflow, facilitant la comparaison et la reproductibilité.

Bénéfices

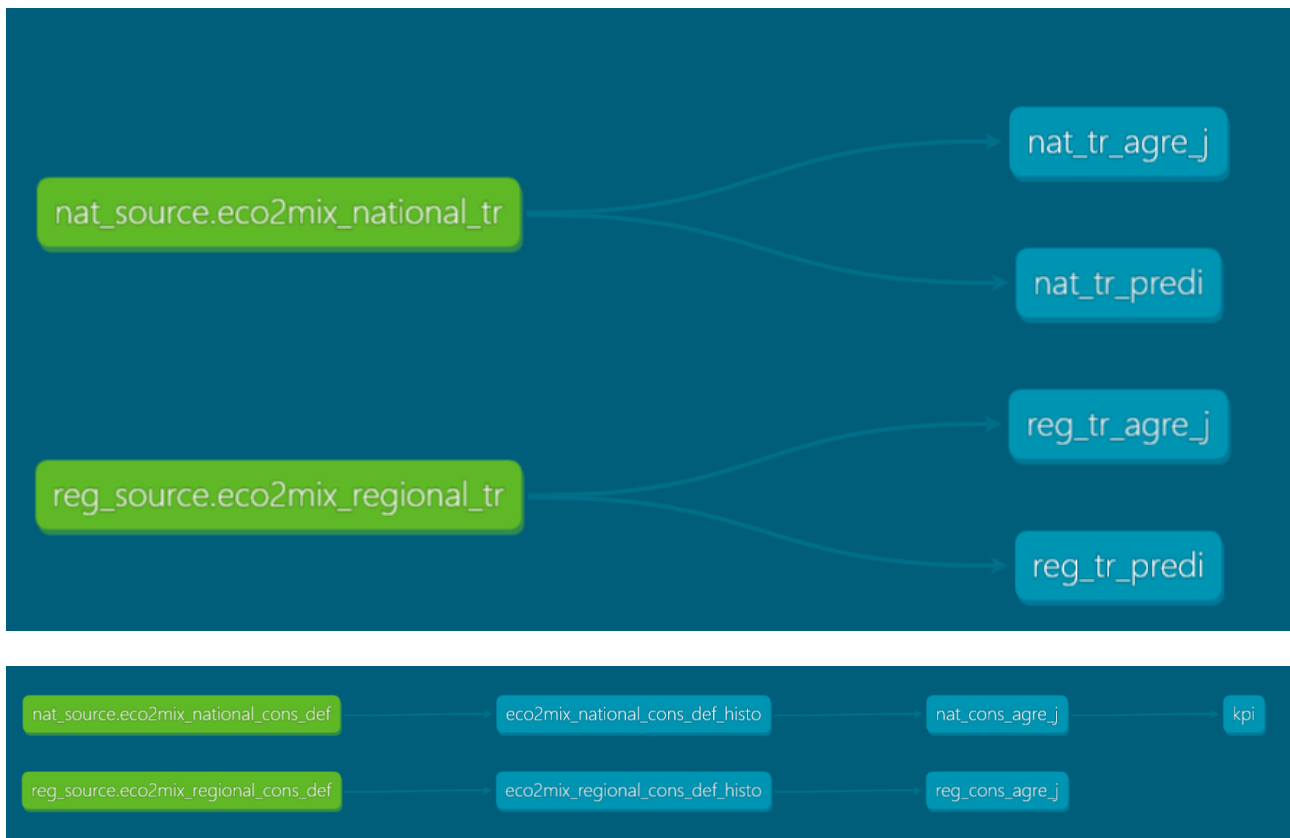
Traçabilité : Chaque expérimentation est historisée (données, hyperparamètres, métriques, artefacts).

Reproductibilité : Possibilité de recharger n'importe quelle version d'un modèle.

Interopérabilité : Le module est agnostique du backend de log (local ou MLflow).

7) Récapitulatif des couches dbt et dbt lineage graph

Couche	Modèle	Matérialisation	Rôle
Silver	eco2mix_national_cons_def_histo	Incrémental	Nettoyage, sélection de colonnes
Silver	eco2mix_regional_cons_def_histo	Incrémental	Idem — périmètre régional
Gold	nat_cons_agre_j	Incrémental	Agrégation journalière nationale
Gold	reg_tr_agre_j	Incrémental	Agrégation journalière régionale
Gold	nat_tr_predi	Table	Features ML (décomposition temporelle)
Gold	kpi	Table	KPIs synthétiques dashboard



4. Base de données

1) Choix technologique : BigQuery (GCP)

Le projet utilise Google BigQuery comme entrepôt de données principal. Ce choix est justifié par plusieurs facteurs :

Volume de données : les séries temporelles demi-horaires sur plusieurs années génèrent des millions de lignes — BigQuery est optimisé pour ce type de charge analytique en lecture.

Intégration GCP : l'import depuis GCS (Google Cloud Storage) est natif, sans infrastructure intermédiaire à maintenir.

Coûts maîtrisés : facturation à la requête (à la quantité de données scannées), idéale pour un projet à usage irrégulier.

Séparation dev/prod : deux datasets distincts (DATASET_ID_DEV / DATASET_ID_PROD) configurés dans dbt profiles.yml.

2) Schéma physique, chargement CSV vers BigQuery

Le chargement des données depuis GCS vers BigQuery est géré par `bigquery_loader.py`. Le mécanisme s'appuie sur un job de chargement asynchrone avec polling côté BigQuery.

```

def load_csv_to_bigquery(
    bq_client: bigquery.Client,
    uri: str,
    table_id: str,
    schema: list[bigquery.SchemaField] | None = None,
    write_disposition: str = bigquery.WriteDisposition.WRITE_TRUNCATE,
    field_delimiter: str = CSV_DELIMITER_SEMICOLON,
) -> bool:
  
```



```

"""
Charge un fichier CSV depuis GCS vers BigQuery.
"""
job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    schema=schema,
    autodetect=schema is None,
    write_disposition=write_disposition,
    field_delimiter=field_delimiter,
    null_markers=NULL_MARKERS,
)

log.info("Chargement de {} -> {} ...", uri, table_id)
load_job = bq_client.load_table_from_uri(uri, table_id, job_config=job_config)

while not load_job.done():
    log.info(" Statut : {} ...", load_job.state)
    time.sleep(5)
    load_job.reload()

if load_job.errors:
    log.error("Erreurs pour {} : {}", uri, load_job.errors)
    return False

table = bq_client.get_table(table_id)
log.info("Succes - {} lignes total dans {}", table.num_rows, table_id)
return True

```

La stratégie WRITE_TRUNCATE écrase la table à chaque run, ce qui garantit la cohérence des données brutes (pas d'accumulation de doublons). Le fallback automatique vers un schéma tout-STRING assure la résilience : si l'inférence de type échoue, le chargement se fait quand même et les conversions de type sont reportées à la couche dbt.

3) Traçabilité des chargements

Une table de suivi `_loaded_files` enregistre chaque fichier chargé avec horodatage, permettant un audit des ingestions :

```

def mark_file_as_loaded(client: bigquery.Client, filename: str):
    """
    Enregistre le fichier dans la table de suivi apres un chargement reussi.
    """
    table_id = f"{PROJECT_ID}.{DATASET_ID}._loaded_files"
    rows = [
        {
            "filename": filename,
            "loaded_at": datetime.now(timezone.utc).isoformat(),
        }
    ]

```

```

schema = [
    bigquery.SchemaField("filename", "STRING"),
    bigquery.SchemaField("loaded_at", "TIMESTAMP"),
]
table = bigquery.Table(table_id, schema=schema)
client.create_table(table, exists_ok=True)

errors = client.insert_rows_json(table_id, rows)
if errors:
    log.warning("Impossible d'enregistrer {} dans le suivi : {}", filename,
errors)

```

4) Configuration dbt

La séparation des environnements dev et prod est configurée dans dbt via des variables d'environnement, sans hard-code de credentials. Cette séparation est réalisée dans le fichier `profiles.yml`

```

mix_energy_profile:
  target: dev
  outputs:
    dev:
      type: bigquery
      method: service-account
      project: "{{ env_var('PROJECT_ID') }}" # BigQuery project ID
      schema: "{{ env_var('DATASET_ID_DEV') }}" # BigQuery dataset/schema to use
for development
      keyfile: "{{ env_var('GOOGLE_APPLICATION_CREDENTIALS') }}" # Path to the
service account key file
      threads: 1
      timeout_seconds: 300
      location: "{{ env_var('LOCATION') }}"
    prod:
      type: bigquery
      method: service-account
      project: "{{ env_var('PROJECT_ID') }}" # BigQuery project ID
      schema: "{{ env_var('DATASET_ID_PROD') }}" # BigQuery dataset/schema to use
for production
      keyfile: "{{ env_var('GOOGLE_APPLICATION_CREDENTIALS') }}" # Path to the
service account key file
      threads: 1
      timeout_seconds: 300
      location: "{{ env_var('LOCATION') }}"

```

5) Schémas des tables

Voici un exemple de schéma de table bronze et gold présentes dans BigQuery :

Table eco2mix_nationa_cons_def :

Field name	Type	Mode
perimetre	STRING	NULLABLE
nature	STRING	NULLABLE
date	DATE	NULLABLE
heure	STRING	NULLABLE
date_heure	TIMESTAMP	NULLABLE
consommation	FLOAT	NULLABLE
prevision_j1	INTEGER	NULLABLE
prevision_j	INTEGER	NULLABLE
fioul	FLOAT	NULLABLE
charbon	FLOAT	NULLABLE
gaz	FLOAT	NULLABLE
nucleaire	FLOAT	NULLABLE
eolien	FLOAT	NULLABLE
solaire	FLOAT	NULLABLE
hydraulique	FLOAT	NULLABLE
pompage	FLOAT	NULLABLE
bioenergies	FLOAT	NULLABLE
ech_physiques	FLOAT	NULLABLE
taux_co2	FLOAT	NULLABLE
ech_comm_angleterre	FLOAT	NULLABLE
ech_comm_espagne	FLOAT	NULLABLE
ech_comm_italie	FLOAT	NULLABLE
ech_comm_suisse	FLOAT	NULLABLE
ech_comm_allemagne_belgique	FLOAT	NULLABLE
fioul_tac	FLOAT	NULLABLE
fioul_cogen	FLOAT	NULLABLE
fioul_autres	FLOAT	NULLABLE
gaz_tac	FLOAT	NULLABLE
gaz_cogen	FLOAT	NULLABLE
gaz_ccg	FLOAT	NULLABLE
gaz_autres	FLOAT	NULLABLE
hydraulique_fil_eau_eclusee	FLOAT	NULLABLE
hydraulique_lacs	FLOAT	NULLABLE
hydraulique_step_turbinage	FLOAT	NULLABLE
bioenergies_dechets	FLOAT	NULLABLE
bioenergies_biomasse	FLOAT	NULLABLE
bioenergies_biogaz	FLOAT	NULLABLE

Table nat_tr_agre_j :

Field name	Type	Mode
perimetre	STRING	NULLABLE
nature	STRING	NULLABLE
date	DATE	NULLABLE
annee	INTEGER	NULLABLE
mois	INTEGER	NULLABLE
jour	INTEGER	NULLABLE
production	FLOAT	NULLABLE
consommation	FLOAT	NULLABLE
prevision_j1	FLOAT	NULLABLE
prevision_j	FLOAT	NULLABLE
fioul	FLOAT	NULLABLE
charbon	FLOAT	NULLABLE
gaz	FLOAT	NULLABLE
nucleaire	FLOAT	NULLABLE
eolien	FLOAT	NULLABLE
solaire	FLOAT	NULLABLE
hydraulique	FLOAT	NULLABLE
pompage	FLOAT	NULLABLE
bioenergies	FLOAT	NULLABLE
ech_physiques	FLOAT	NULLABLE
taux_co2	FLOAT	NULLABLE

6) Conformité RGPD

Les données traitées sont exclusivement des séries temporelles de production électrique agrégées à l'échelle nationale et régionale. Aucune donnée à caractère personnel n'est collectée ni stockée.

Les mesures de conformité suivantes ont été appliquées :

- ▶ Authentification par compte de service GCP avec permissions minimales (lecture GCS + écriture BigQuery uniquement).
- ▶ Credentials jamais committés dans le repo (gérés via variables d'environnement, fichier .env.copy fourni à titre indicatif).
- ▶ Les logs Airflow ne tracent pas les valeurs de données, uniquement les métadonnées (noms de fichiers, nombres de lignes, statuts).

5. Mettre à disposition les données via une API

1) Architecture REST : FastAPI

L'API est développée avec FastAPI, qui génère automatiquement une documentation OpenAPI (Swagger UI) accessible en ajoutant « /docs » à la fin de l'url. Elle expose les données BigQuery en lecture seule et les prédictions de consommation :

Méthode	Endpoint	Rôle	Paramètres
GET	/health	Statut de l'API	—
GET	/tables	Liste des tables disponibles	—
GET	/tables/{name}/columns	Schéma d'une table	table_name
GET	/tables/{name}	Requête de données	columns, filters, limit
POST	/predict/national	Prédiction conso nationale	—
POST	/predict/region	Prédiction conso régionale	code_insee_region

Voici une vue de la page :

default ^

GET /health Health

GET /tables List Tables

GET /tables/{table_name}/columns Get Table Columns

GET /tables/{table_name} Query Table

POST /predict/national Predict Nat

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  "http://localhost:8888/predict/national" \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8888/predict/national
```

Server response

Code

Details

200

Response body

```
47202.83926266747
```

2) Schémas Pydantic

Tous les corps de réponse sont modélisés avec Pydantic dans le fichier schemas.py pour une validation et une sérialisation strictes :

```
FilterOperator = Literal[
    "eq",
    "ne",
    "lt",
    "lte",
    "gt",
    "gte",
    "contains",
    "in",
    "is_null",
    "not_null",
]

class FilterClause(BaseModel):
    model_config = ConfigDict(extra="forbid")

    field: str = Field(..., min_length=1)
    operator: FilterOperator
    value: Any | list[Any] | None = None
```

```
class QueryResponse(BaseModel):
    project_id: str
    dataset_id: str
    table_name: str
    selected_columns: list[str]
    filters: list[FilterClause]
    limit: int
    row_count: int
    rows: list[dict[str, Any]]
```

L'utilisation de Pydantic avec `extra='forbid'` sur `FilterClause` empêche les injections de champs inattendus dans les filtres. Les opérateurs sont définis comme un `Literal` exhaustif (`FilterOperator`), ce qui génère automatiquement la validation dans l'interface Swagger. Toute requête malformée reçoit une réponse HTTP 422 avec le détail du champ invalide.

3) Endpoint de requête générique

L'endpoint GET /tables/{table_name} accepte des paramètres de sélection de colonnes et des filtres JSON dynamiques :

```
@app.get("/tables/{table_name}", response_model=QueryResponse)
def query_table(
    table_name: str,
    columns: str | None = Query(
        default=None, description="Comma-separated list of columns"
    ),
    filters: str | None = Query(
        default=None,
        description="JSON array of filter clauses, e.g.
[{'field': 'date', 'operator': 'gte', 'value': '2024-01-01'}]",
    ),
    limit: int | None = Query(default=None, ge=1),
    service: BigQueryDatasetService = Depends(get_service),
) -> dict[str, Any]:
    parsed_columns = _parse_columns(columns)
    parsed_filters = _parse_filters(filters)

    try:
        return service.query_table(
            table_name,
            columns=parsed_columns,
            filters=parsed_filters,
            limit=limit,
        )
    except UnknownTableError as exc:
        raise HTTPException(status_code=404, detail=str(exc)) from exc
    except UnknownColumnError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except InvalidFilterError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except BigQueryServiceError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
```

4) Sécurisation et CORS

L'API est configurée avec un middleware CORS restrictif, n'autorisant que les origines définies dans les settings :

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=list(settings.allowed_origins),
    allow_credentials=True,
    allow_methods=["GET", "OPTIONS"],
    allow_headers=["*"],
)
```

La restriction des méthodes HTTP à GET et OPTIONS garantit que l'API est strictement en lecture seule — aucune modification des données BigQuery n'est possible depuis l'extérieur. Les origines autorisées sont injectées via les settings (variables d'environnement), ce qui permet de paramétrer différemment les environnements dev, préprod et prod sans toucher au code.

5) Dashboard Streamlit

Récupération des données via l'API FastAPI

Le module front-end Streamlit utilise un client dédié pour interroger l'API FastAPI :

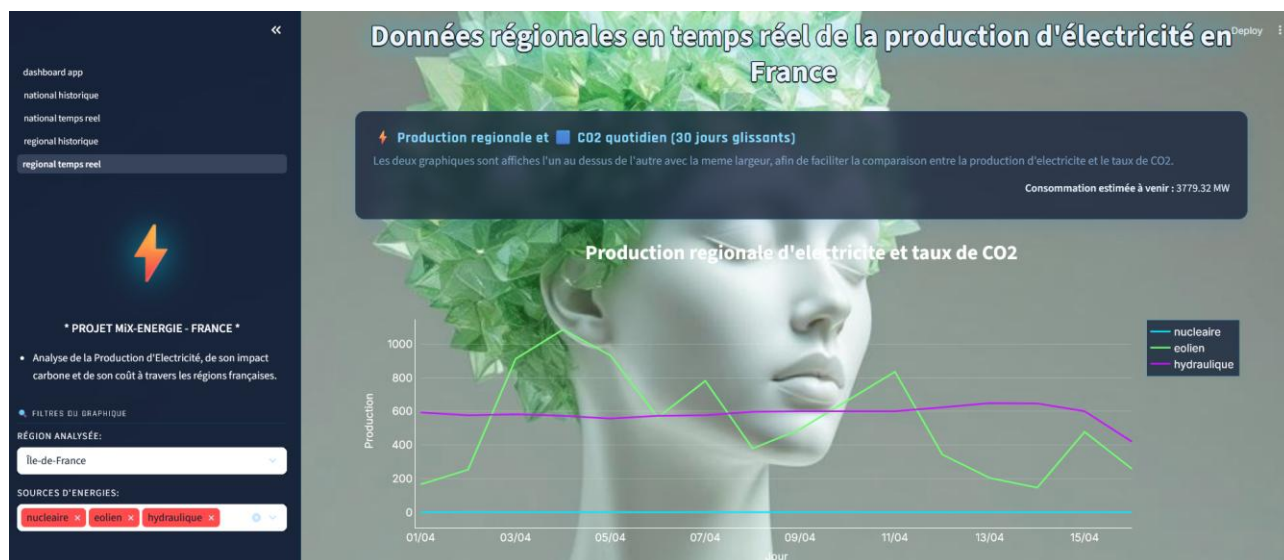
```
# Extrait de data_api_client.py
import requests

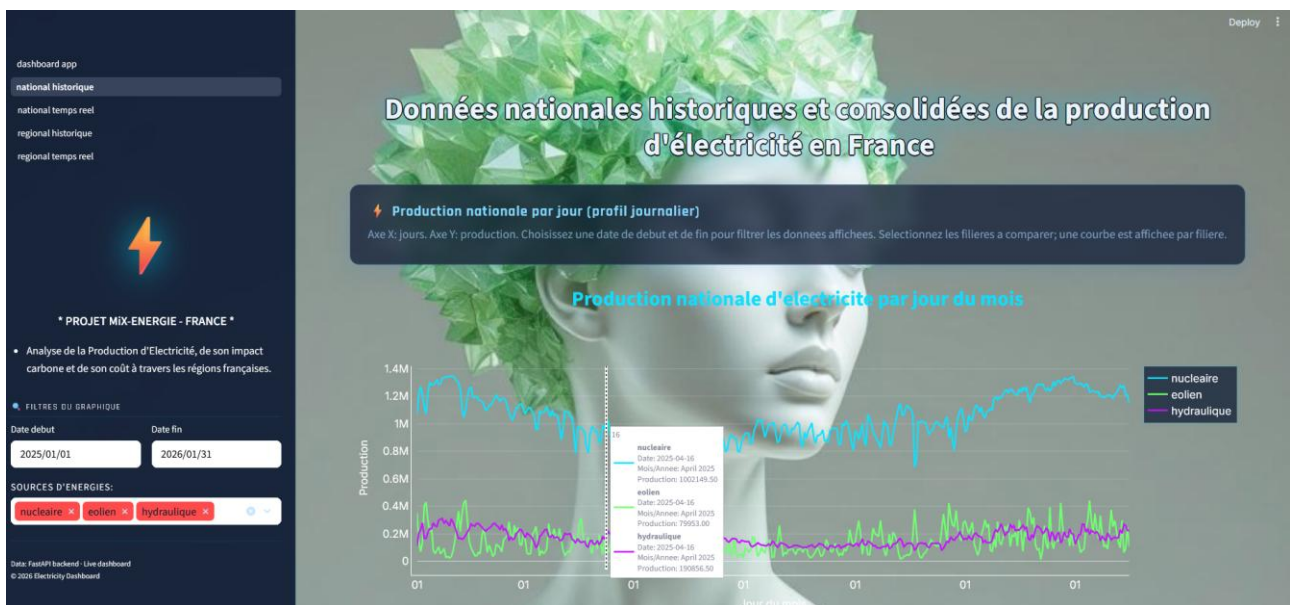
class FastAPIClient:
    def __init__(self, base_url):
        self.base_url = base_url

    def query_table(self, table_name, filters=None, limit=None):
        params = {"limit": limit or 1000}
        if filters:
            params["filters"] = json.dumps(filters)
        response = requests.get(f"{self.base_url}/tables/{table_name}",
                                params=params)
        return response.json()
```

Streamlit utilise ce client pour envoyer des requêtes HTTP à l'API (par défaut sur <http://localhost:8888>) et récupérer les données sous forme de dictionnaire ou de DataFrame.

Exemples d'affichage dans Streamlit





Résumé :

Streamlit interroge l'API FastAPI via un client HTTP, puis affiche les résultats sous forme de graphiques interactifs ou de tableaux.

6. Déploiements

L'application est conteneurisée pour un déploiement reproductible, avec plusieurs Dockerfile. Un pour la FastAPI, un pour streamlit et un pour airflow contenant les fichiers d'ingestions et de dbt en y ajoutant des volumes.

```
volumes:
- ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
- ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
- ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
- ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
- ../ingest_dbt/src:/opt/project/src:ro
- ../dbt:/opt/project/dbt
- ../data/gcp:/opt/project/data/gcp:ro
```

Un fichier Makefile a été créé afin de faciliter le déploiement des images. Les informations pour lancer les builds sont indiquées dans le README.md

```
> - build_local_fastapi : construit en local l'image docker de l'application FastAPI
> - build_local_airflow : construit en local l'image docker de l'application AirFlow
> - build_local_streamlit : construit en local l'image docker de l'application Streamlit
```

7. Infrastructure as Code avec Terraform

Présentation de Terraform

Terraform est un outil d'Infrastructure as Code (IaC) open source qui permet de décrire, provisionner et gérer l'infrastructure cloud de manière déclarative. Grâce à Terraform, l'ensemble de l'architecture nécessaire au projet (projets GCP, services, buckets, etc.) est versionné et reproductible, facilitant le déploiement sur différents environnements (développement, préproduction, production).

Exemple de configuration

Déclaration du provider et du backend

```
# provider.tf
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "5.24.0"
    }
  }
  backend "local" {}
}

provider "google" {
  project = var.project_id
}
```

Ici on indique à Terraform d'utiliser le provider Google Cloud et de stocker l'état localement.

Création d'un projet GCP et activation des APIs

```
# main.tf
resource "google_project" "mix_energie_gcp" {
  name           = var.project_name
  project_id     = var.project_id
  org_id         = var.org_id
  billing_account = var.billing_account
  auto_create_network = true
}

resource "google_project_service" "compute" {
```

```

project = google_project.mix_energie_gcp.project_id
service = "compute.googleapis.com"
depends_on = [google_project.mix_energie_gcp]
}

```

Explication : Ce code crée un projet Google Cloud et active l'API Compute Engine, indispensable pour utiliser les VM.

Utilisation des variables et de l'environnement

```

# auto.tfvars
billing_account      = "${env.BILLING_ACCOUNT}"
org_id              = "${env.ORG_ID}"
project_id          = "${env.PROJECT_ID}"
project_name        = "${env.PROJECT_NAME}"
project_bucket_name = "${env.PROJECT_BUCKET_NAME}"
gcp_user_email      = "${env.GCP_USER_EMAIL}"

```

Explication : Les paramètres sensibles ou susceptibles de changer sont injectés via des variables d'environnement, ce qui évite de stocker des secrets dans le code source.

Bonnes pratiques et points importants

- **Versionnement** : Les fichiers d'état (terraform.tfstate) et les secrets ne sont jamais versionnés.
- **Reproductibilité** : Toute l'infrastructure peut être reconstruite à l'identique sur un nouvel environnement.
- **Sécurité** : Les identifiants et secrets sont gérés via des variables d'environnement et non dans le code.
- **Automatisation** : Un simple terraform init puis terraform apply permet de déployer toute l'architecture cloud nécessaire au projet.

8. Conclusions

Le projet Mix-Energie s'articule autour d'un pipeline de données complet, allant de la collecte à la mise à disposition de données exploitables.

Dans un premier temps, les données sont collectées depuis diverses API publiques, puis ingérées et stockées dans Google Cloud Storage, qui joue le rôle de lac de données brutes. Ces données sont ensuite chargées dans BigQuery, où elles alimentent des tables dites "brutes", reflétant fidèlement la structure d'origine des sources.

Dans un second temps, ces tables brutes font l'objet de transformations successives : nettoyage des données, gestion des valeurs manquantes ou incohérentes, et agrégations diverses. L'objectif est de produire des datasets fiables, normalisés et directement consommables par d'autres équipes ou utilisateurs finaux.

L'ensemble de ce pipeline est entièrement automatisé grâce à l'orchestrateur Apache Airflow. Celui-ci permet de définir des workflows sous forme de graphes de tâches, en s'assurant que chaque étape s'exécute dans le bon ordre, en respectant les dépendances entre elles. Cette approche garantit non seulement la reproductibilité des traitements, mais aussi une observabilité accrue du pipeline : en cas d'erreur, il est possible d'identifier rapidement l'étape défaillante. Cette architecture est également pensée pour être évolutive, afin d'accueillir facilement de nouveaux datasets à l'avenir sans remettre en cause l'existant.

Enfin, l'intégration de Terraform pour la gestion de l'infrastructure cloud permet d'automatiser et de versionner le déploiement de tous les composants nécessaires (projets GCP, services, buckets, etc.). Cette démarche Infrastructure as Code assure la reproductibilité, la sécurité et la portabilité de l'environnement, tout en facilitant la collaboration et le passage à l'échelle du projet.

9. Axes d'améliorations

- ▶ Ingestion automatique d'une source de type météo pour améliorer les prédictions des productions par type de centrale. Permet d'avoir un historique et des prédictions des relevés météo permettant un meilleur apprentissage du modèle de prédiction.
<https://open-meteo.com/en/docs>
- ▶ Ingestion automatique d'une source sur la qualité de l'air permettant de voir s'il existe une corrélation entre la production d'électricité et de la qualité de l'air dans une région donnée.
Les données météo peuvent également permettre d'être plus pertinent sur un éventuel lien entre la qualité de l'air et la production d'énergie. (Vents, pluie peuvent avoir un impact sur les particules présente dans l'air)
<https://admindata.atmo-france.org/api/doc/v2>
- ▶ Mise en place de table d'historisation des données en temps réelles. Cette mise en place permettrait de pouvoir comparer les écarts entre les données en temps réelle et les données renseignées par la suite dans les tables consolidées.
- ▶ Mise en production du pipeline afin que l'ensemble des fonctionnalités puissent être accessible en dehors de la VM GCP.